현업 질문 응답을 위한 AI Agent 기반 Text-to-SQL 미니 프로젝트

TC 업무 지원을 위한 자연어 질의 시스템

- 2026년 4월 24일
- 설비SW팀
- 현재 구현 기준

발표 순서

오늘의 발표 흐름

PART 01
TC 업무 소개
환경, 역할, 운영 구조

PART 02
프로젝트 선정 배경
문제 → 원인 → 기회

PART 03
전체 Agent 구조
DB Agent 중심 설명

PART 04
Text-to-SQL 핵심 구현
Planner → SQL → 응답

PART 05
Schema Linker
TF-IDF 기반 스키마 탐색

PART 06
SQL 생성 · 검증
Generator / Validator / Refiner

PART 07
성능 최적화 기법
속도와 정확도의 균형

PART 08
향후 로드맵
확장 계획 및 마무리

PART 01

TC 시스템 운영 환경

서버 장애 = 라인 장애. 안정성이 핵심입니다.

🔧 운영 구조

- FW 구조 기반 시스템 운영 환경
- 서버 이중화 구성 — 단일 장애점 없음
- everRun 등 Failover 라이선스 운용 중
- 서버 이상 감지 자동화 적용

⚠️ 운영 민감도

서버 장애 → 라인 장애로 직결
가용성 확보가 최우선 과제

시스템 안정성 구조

FW **FW 기반 구조**
Firmware 제어 시스템

이중화 **서버 이중화 (HA)**
Active-Standby 구성

FO **Failover (everRun)**
자동 장애 전환 라이선스

PART 01

담당 역할 및 경험

주요 역할

- **E 직군** — 공정 업무 병행
- 현업 요청 기능 검토 및 개발 (**약 3년**)
- 설비 동작 관련 **VOC 대응**
- TC DB 질문 응대 업무

업무 흐름

1. 공정 운영 & 모니터링
2. 현업 VOC 접수 및 분류
3. DB 조회 및 기능 검토
4. **AI Agent 개발 (미니 프로젝트)**

PART 02

왜 Text-to-SQL인가

문제 → 원인 → 기회

PROBLEM

반복되는 현업 질문

- VOC의 상당수가 TC DB 정보 요청
- 대시보드(Spotfire, Drone)는 있으나 table/column 의미를 현업이 이해하기 어려움
- 재문의 발생 → 담당자 부담 증가

CAUSE

접근성의 한계

- 서버 예측 과제 → 이미 이중화·라이선스로 안정성 확보
- SQL 직접 작성이 어려운 현업 담당자
- 개발 숙련도에 따라 답변 품질 차이 발생

OPPORTUNITY

Text-to-SQL POC

- 자연어로 질문 → DB에서 자동 조회
- 답변 품질 표준화

→ 실제 TC 업무에 직접 연결되는 주제

PART 03

전체 AI Agent 시스템 구조

오늘 발표는 DB Agent에 집중합니다.

PART 03

Text-to-SQL이란?

쉽게 설명하면

사람이 말로 질문하면, 시스템이 그 뜻을 이해해서
SQL로 바꿔 DB에서 답을 찾아주는 기술입니다.

비유

도서관 사서에게 "최근 불량이 많은 장비 알려줘"라고 하면
사서가 관련 기록을 찾아오는 것과 같습니다.

왜 필요한가?

- SQL 몰라도 DB 정보 조회 가능
- 질문 방식에 따른 답변 품질 차이 해소
- 반복 질의 대응 자동화

처리 흐름

사용자 입력
"최근 한 달 불량 TOP 5 장비 알려줘"

↓ 의미 파악

SQL 생성

```
SELECT eq_id, COUNT(*) AS cnt
FROM defects WHERE ...
ORDER BY cnt DESC LIMIT 5
```

↓ DB 실행 → 자연어 답변

최종 응답
"지난 달 불량이 가장 많은 장비는 EQ-A01 (43건)입니다."

PART 04

Planner — 질문 분류 구조

어떤 에이전트로 보낼지 결정하는 관문

Tier 3 구조 (현재 구현)

TIER 1

Prefilter
명확히 DB 무관한 질문 조기 제거 — 빠른 응답

↓

TIER 2

Classifier
bge-m3 embedding + Logistic Regression
질문을 벡터로 변환 후 분류

↓

TIER 3

Fallback
신뢰도(confidence) 낮을 때만 호출
T20 routing → DB Agent 선택

초등학생 설명

질문을 **숫자 좌표**로 바꾸고,
어떤 종류의 질문에 가장 가까운지 계산합니다.
비슷한 무리끼리 나누는 **똑똑한 분류기**예요.

머신러닝 포인트

요소	역할
bge-m3 embedding	질문 → 고차원 벡터 변환
Logistic Regression	벡터 기반 intent 분류
confidence score	확신도 낮으면 LLM으로 위임

PART 05

Schema Linker — 관련 테이블 탐색

전체 스키마 중 관련 있는 부분만 좁혀냅니다.

초등학생 설명

책의 **목차**를 보고 가장 관련 있는 장을 찾는 것처럼,
질문에서 중요한 단어를 뽑아 어떤 테이블 설명과
가장 비슷한지 **점수로 계산**합니다.

예시

질문: **"최근 불량이 많은 장비 알려줘"**

→ "불량", "장비" 단어 추출
→ **defect_log** 테이블 설명과 유사도 **높음** ✓
→ user_account 테이블 설명과 유사도 **낮음** ✗

처리 단계

1. 질문 → 키워드 추출
2. SchemaStore에서 TF-IDF 검색
3. 상위 후보 table/column 선정
4. Schema subset 구성 → SQL Generator로 전달

TF-IDF란?

개념	의미
TF (Term Frequency)	단어가 문서에 얼마나 자주 나오는가
IDF (Inverse Doc Freq)	희귀한 단어일수록 더 중요하게 취급
활용	질문과 스키마 설명의 유사도 계산

★ 머신러닝 기반 구조 — LLM 없이 동작

좁혀지는 구조

전체 스키마 (100개+) → 후보 10~20개 → Schema Subset

PART 06

SQL Generator

질문·스키마·예시를 받아 실행 가능한 SQL을 만들어냅니다.

입력 / 출력

구분	내용
입력	자연어 질문 + Schema subset + Few-shot 예시
처리	GPT + Gemma3 병렬 호출 → Ensemble 결합
출력	최적 SQL 1개 선택 → Validator로

왜 Ensemble을 넣었는가 — 장점 3가지

1. **정확도 보강** 핵심
   두 모델이 독립적으로 같은 SQL을 만들면 맞을 가능성이 높습니다. 한 모델이 실수해도 다른 모델이 잡아줍니다.
2. **신뢰도 신호 자동 생성**
   일치 → confidence 상승, 그냥 사용
   불일치 → Active Learning 큐로 보내 Golden 데이터 보강 — **비용 없이 저신뢰 케이스 자동 선별**
3. **추가 시간 없음**
   `asyncio.gather` 로 병렬 호출 — **max(GPT, Gemma3) ≈ GPT 단독**과 같습니다. 비동기라 사실상 무료.

Ensemble 동작 흐름

Schema Subset
+ Few-shot

↓ 동시 호출 (asyncio.gather)

GPT SQL_A	Gemma3 SQL_B

↓ 비교

A = B 일치 confidence ↑ 그냥 사용	**A ≠ B 불일치** Active Learning 큐 → Golden 보강

⚠️ 단점 및 대응

- LLM API 호출 비용 **2배**
- Gemma3 장애 시 Ensemble 전체 영향 → `enabled: false` 로 단독 모드 전환 가능하게 설계

PART 06

Validator — SQL 안전성 검증

생성된 SQL이 안전하고 실행 가능한지 사전에 확인합니다.

검증 항목

항목	내용
문법 검증	SQL 구문 유효성 확인
위험 쿼리 차단	DROP, DELETE, UPDATE 등 차단
실행 가능성	테이블·컬럼 존재 여부 사전 확인

초등학생 설명

시험 답안을 제출하기 전에
맞춤법 검사 + 위험한 내용 없는지 먼저 확인하는 단계입니다.

Validator 통과/실패 분기

SQL Generator 출력

↓ Validator 검증

✓ 통과
DB Execution으로

✗ 실패
Refiner 호출

★ DB에 실제 쿼리가 닿기 전 마지막 방어선

PART 06

DB 실행 & 조건부 Refiner

한 번에 맞으면 통과 — 문제가 있을 때만 다시 고칩니다.

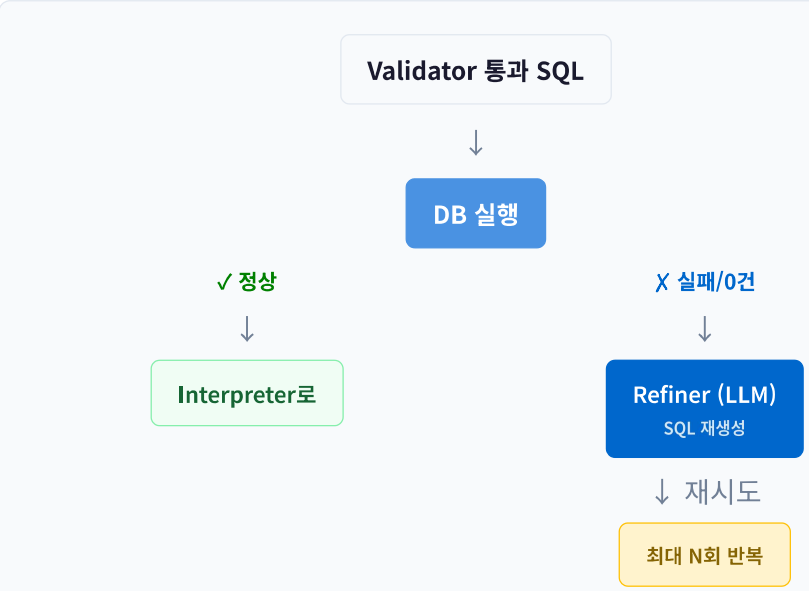
초등학생 설명

시험을 제출하면 바로 채점합니다.
맞으면 그냥 통과! 틀렸을 때만
다시 풀어보는 기회(Refiner)가 주어집니다.

Refiner 호출 조건 조건부 호출

상황	Refiner 호출?
정상 실행, 결과 있음	✗ 호출 안 함
SQL validation 실패	✓ 호출
DB execution 오류	✓ 호출
결과 0건	✓ 호출

실행 분기 흐름



PART 06

Interpreter · Synthesizer · Final Answer

DB 숫자를 사람의 말로 바꾸는 마지막 단계

초등학생 설명

DB가 **숫자와 표**를 주면,
Interpreter가 **사람 말로 풀어주고**,
Synthesizer가 **최종 문장으로 정리**합니다.

각 단계 역할

단계	하는 일
Interpreter	DB 결과 rows → 자연어 해석
Synthesizer	전체 흐름 취합 → 응답 구성
Final Answer	Synthesizer 출력 → 사용자 전달

응답 생성 흐름

DB 실행 결과
`[(EQ-A01, 43), (EQ-B02, 38), ...]`

↓ Interpreter

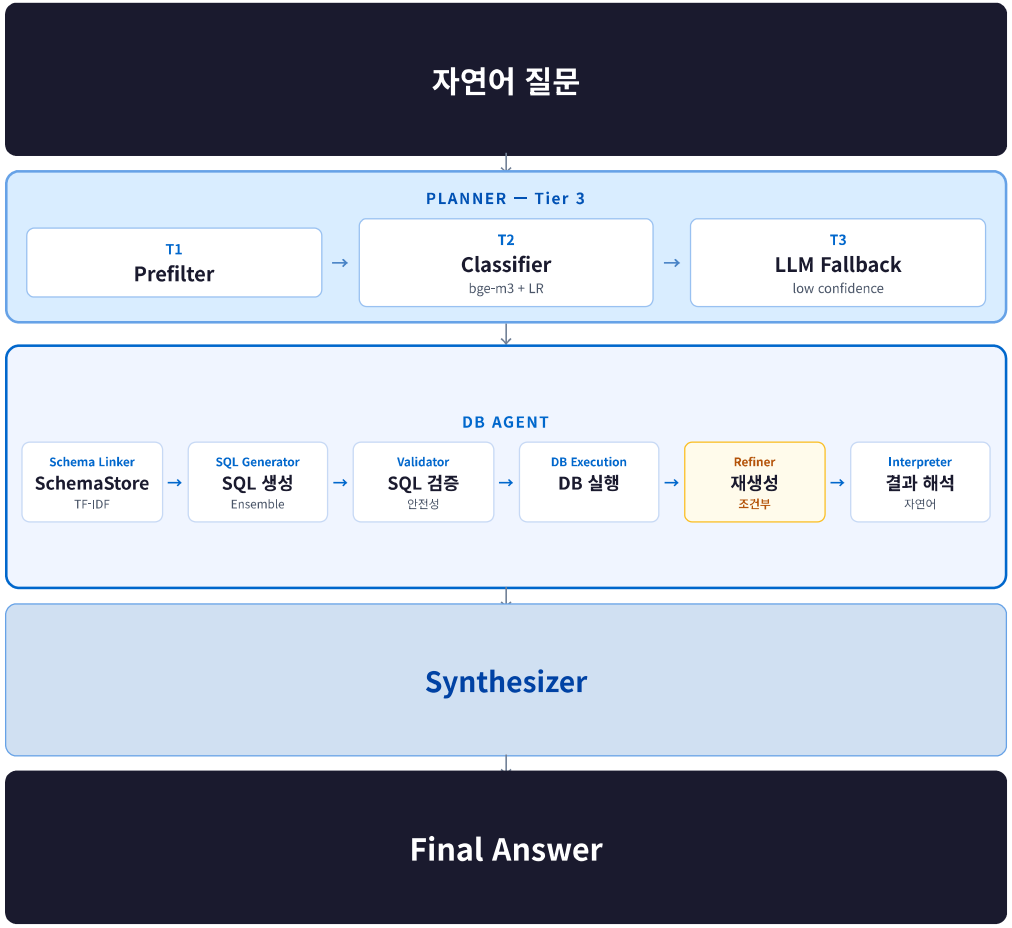
자연어 해석
"EQ-A01이 43건으로 가장 많고, 그 다음은 EQ-B02 38건입니다."

↓ Synthesizer

Final Answer
"지난 달 불량이 가장 많은 장비는 EQ-A01 (43건)입니다. 다음으로..."

현재 구현 기준

DB Agent 전체 파이프라인



- "MSCDA21 설비 CD1번 개발되어있어?"

- "DB를 조회해야겠네, DB Agent 호출해"

- "TC_DCOL_ITEM에서 MSCDA21, CD1 조회"

eqp_id	dcol_item
MSCDA21	CD1

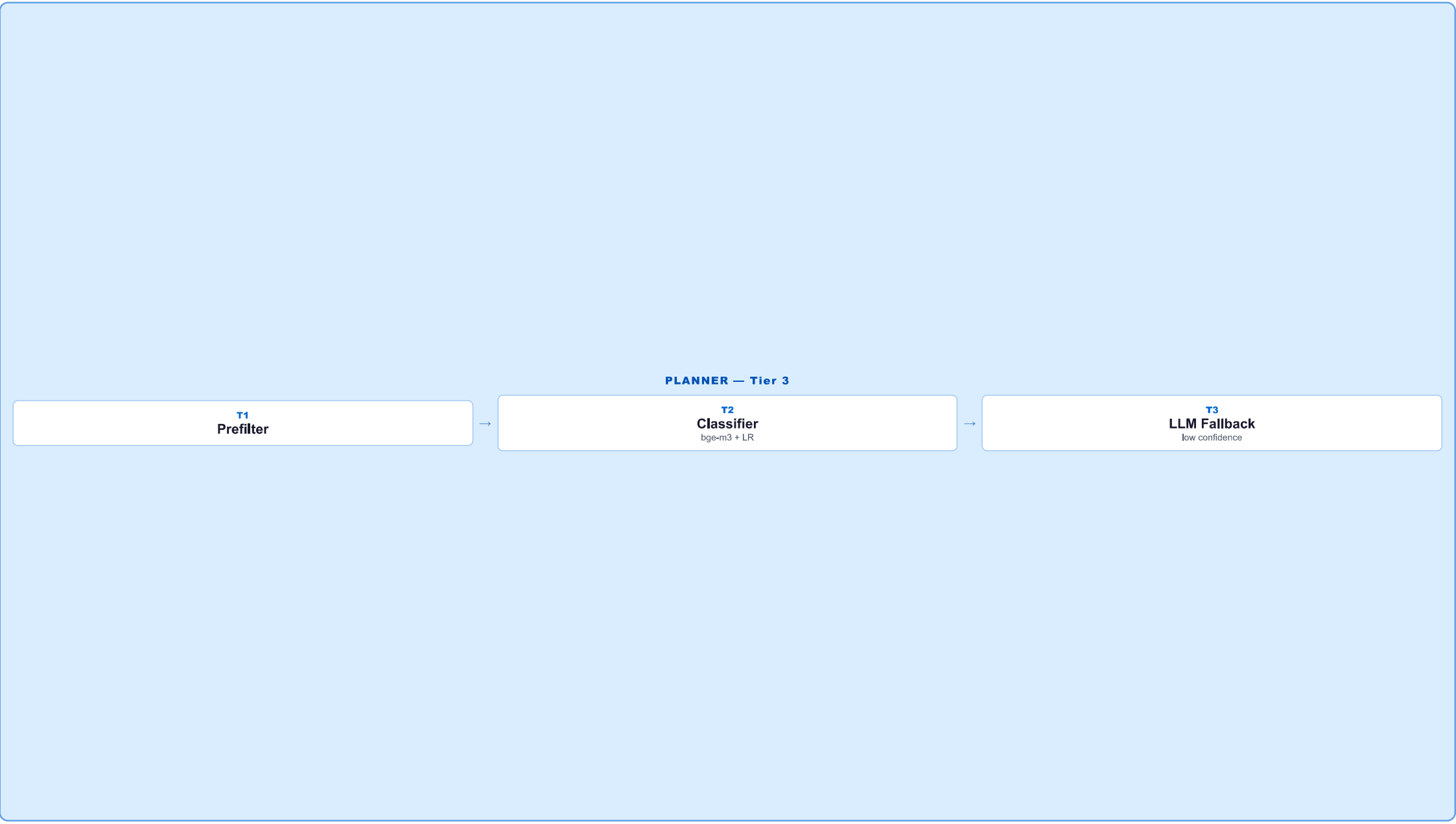
- "SQL 조회됐으니까 개발되어 있다고 응답 구성"

- "MSCDA21에 CD1은 개발되어 있습니다."

자연어 질문

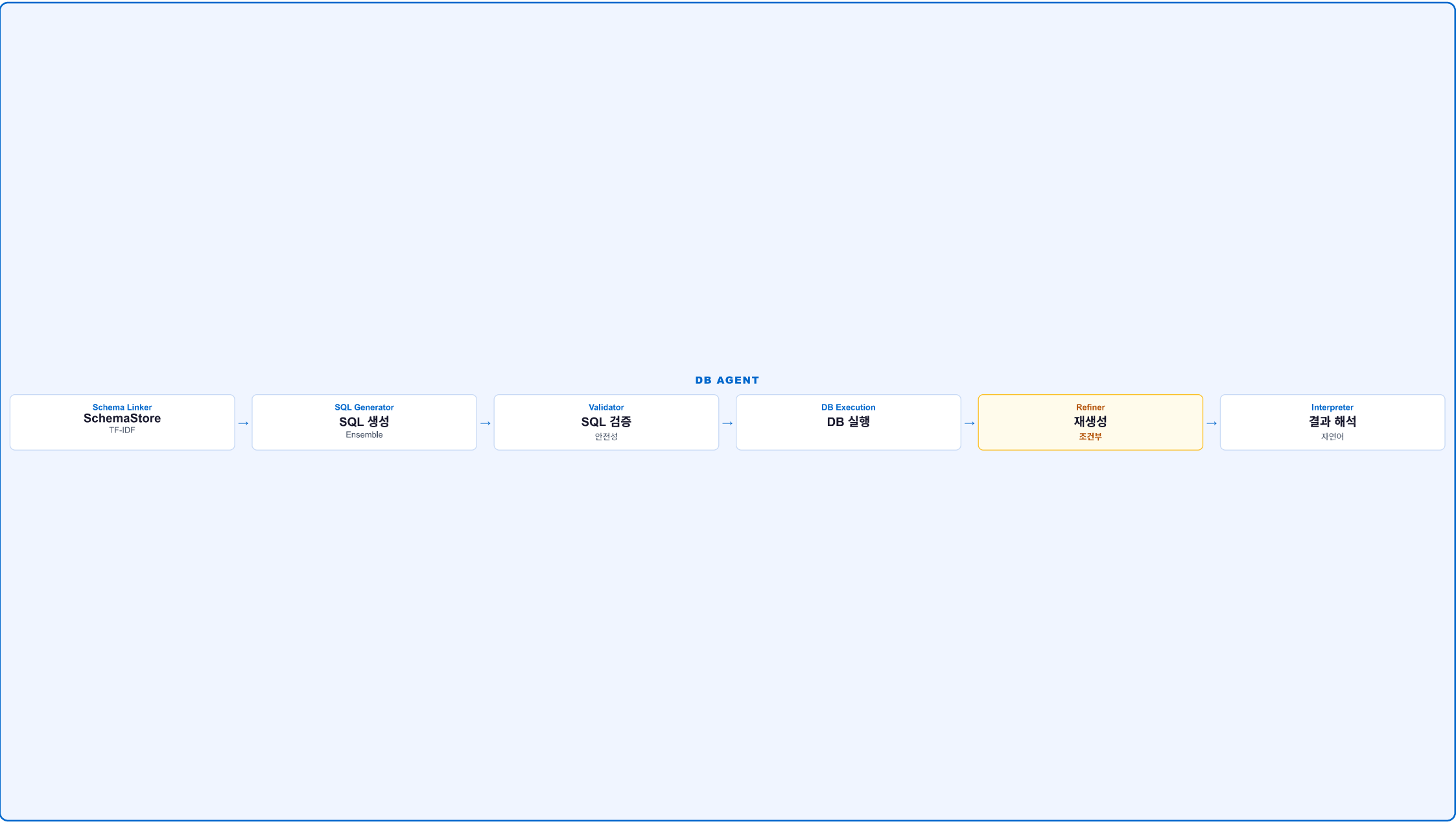
- **"MSCDA21 설비 CD1번 개발되어있어?"**

↓


PLANNER — Tier 3
T1
Prefilter
→
T2
Classifier
bge-m3 + LR
→
T3
LLM Fallback
low confidence

- **"DB를 조회해야겠네, DB Agent 호출해"**

↓


DB AGENT
Schema Linker
SchemaStore
TF-IDF
→
SQL Generator
SQL 생성
Ensemble
→
Validator
SQL 검증
안전성
→
DB Execution
DB 실행
→
Refiner
재생성
조건부
→
Interpreter
결과 해석
자연어

- **"TC_DCOL_ITEM에서 MSCDA21, CD1 조회"**

eqp_id	dcol_item
MSCDA21	CD1

↓

Synthesizer

- **"SQL 조회됐으니까 개발되어 있다고 응답 구성"**

↓

Final Answer

- **"MSCDA21에 CD1은 개발되어 있습니다."**

PART 07

응답 성능 & 품질 향상 기법

왜 이 구조가 실무적으로 의미 있는가

속도 최적화

기법	효과
Prefilter (T18)	무관 질문 조기 차단
Tier 구조 Planner	LLM 호출 최소화
Schema Narrowing	LLM 입력 컨텍스트 축소
조건부 Refiner	불필요한 재생성 방지

품질 향상

기법	효과
bge-m3 + LR Classifier	정확한 intent 분류
TF-IDF Schema Linking	관련 스키마 정밀 선별
Ensemble SQL 생성	단일 모델 대비 정확도↑
Validator + Refiner	오류 SQL 자동 수정

핵심 **LLM 1회 호출**이 아닌 **단계별 ML + LLM 조합**으로 정확도와 속도를 동시에 확보합니다.

PART 08

향후 로드맵

단계적으로 확장 가능한 구조로 설계되었습니다.

PHASE 1 — 품질 개선

Text-to-SQL 고도화

- → Text-to-SQL 개선방안 추가 도입
- → Embedding Model 확산 적용
- → Feedback 구조 도입 (사용자 피드백 반영)

PHASE 2 — 구조 확장

Agent 추가 도입

- → RAG 구조 도입 (Confluence 문서 연계)
- → Splunk Log Agent 도입
- → 두 시스템 모두 API 지원 확인 완료

PHASE 3 — 서비스화

TC 통합 플랫폼

- → Web 개발 및 UI 구성
- → 대시보드 기능 개발
- → 파편화된 TC 기능 통합 (로그 다운로드 등)

마무리

작게 시작했지만, 실제 업무에 바로 연결되는 구조를 만들었습니다.

- 현업 질문 대응 품질 **표준화** 의 첫 걸음
- 자연어 질의로 누구나 **쉽게** TC DB에 접근
- ML + LLM 조합의 **실무 적용** 가능성 입증